

Bitácora

Trabajo Final de PDyTR Raspberry PI - Video “de Interés” a Back End

Objetivo: Tener dos o más “edges” con cámaras web para enviar video “de interés” a un back end para que haga procesamiento más específico.

Comentarios-Requerimientos:

- Construcción de un ambiente distribuido simple, con procesamiento mínimo en los edges (RPi con webcam) y en el back end el procesamiento puede ser tan sencillo como directamente poner un servidor con el streaming de video “de interés”.
- La definición “de interés” al menos por ahora es “video con identificación de movimiento”. Es decir: desde los edges se va a comunicar video al back end solamente cuando se identifique que hay movimiento.
- Cada edge tiene que tener procesamiento local para discriminar/decidir si el video es “de interés” (hay algo que se mueve en el video capturado) o no. Ese procesamiento debe hacerse necesariamente en el edge/en donde está la webcam. En caso de identificar movimiento, se envía/hace streaming del video al back end. En caso de no identificar movimiento no se hace nada más que seguir procesando el video para identificar si hay movimiento o no.
- Al menos un edge debe ser “real”: una RPi con una webcam donde se procese el video y se discrimine si hay movimiento o no. No hay nada virtualizado en cada edge real (RPi).
- Tiene que haber al menos 2 edges. Si todos los edges son RPi, mejor, sino, se pueden “simular/virtualizar” edges en una o más PCs.

IMPORTANTE: como se dejó claro desde la primera conversación (17/11/25 con F. Torrico), la cátedra no necesariamente puede proveer el material y los desarrolladores del proyecto tienen lo necesario para llevarlo adelante. En principio, la cátedra dispone solamente de una webcam. Si bien la cátedra intentará conseguir más material, la disponibilidad del mismo no se puede asegurar de antemano y eventualmente puede ser posible que se disponga del mismo durante un período limitado del año (normalmente el primer semestre, dado que en el segundo semestre varias cátedras necesitan el material que se podría conseguir). Si hay alguna duda acerca de la disponibilidad de materiales, se recomienda que ni siquiera se inicie este Trabajo dado que la factibilidad del desarrollo depende como mínimo del material, tal como queda claro en los Comentarios-Requerimientos enunciados anteriormente. Este trabajo es completamente opcional para el final de la asignatura, no es obligatorio.

Eventualmente, se podría definir la tesina de Licenciatura como continuación de este Trabajo.

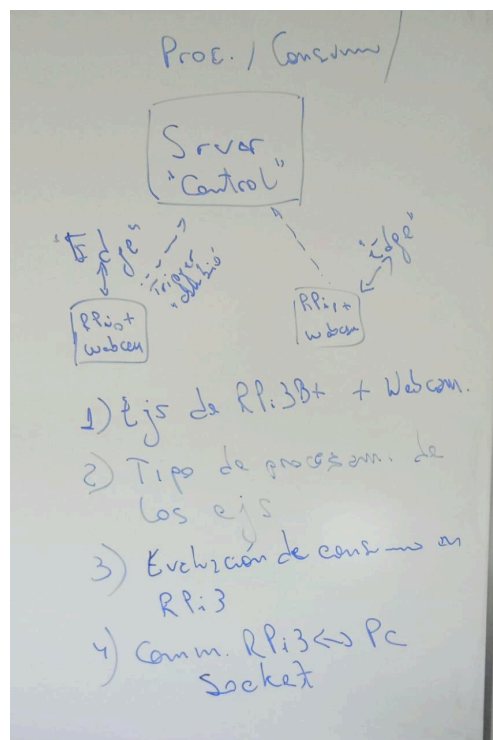
Cada entrada/comentario de esta bitácora se debe hacer con este formato (como está a continuación):

- Una línea horizontal
- Fecha y autor de la entrada/comentario

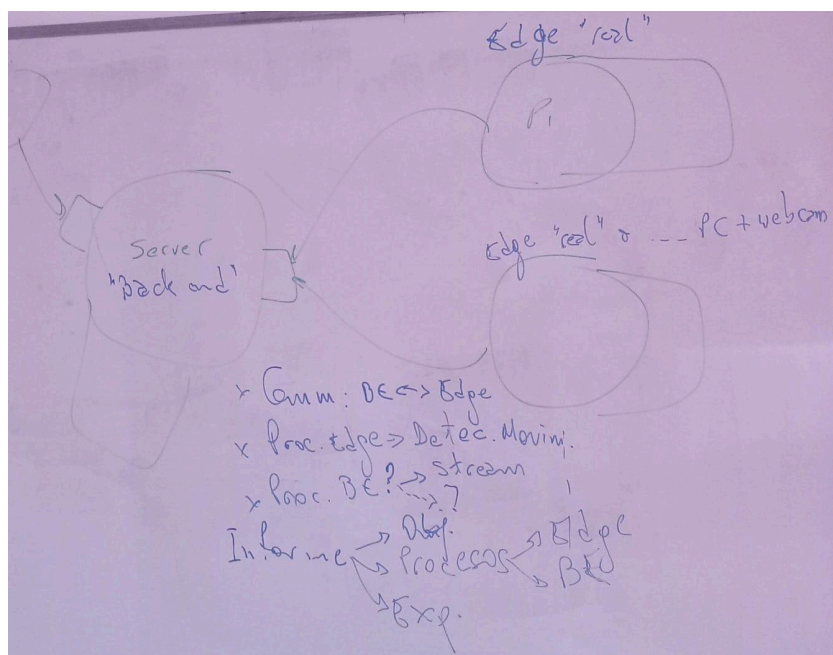
12/12/25 - Profesor

Al día de la fecha, se tuvieron dos reuniones: 1) 17/11/25 con F. Torrico y 2) 10/12/25 con ambos estudiantes (Torrico-Ricciardi). Dado que aunque se recomendó que enviaran la foto de lo conversado en ambas fechas de reuniones esas fotos no se enviaron, las incorporo ahora en directamente en esta Bitácora (a futuro todo lo conversado debe ser volcado en esta Bitácora por parte de los estudiantes):

17/11/25:



10/12/25:



Se envía por Ideas la URL de la carpeta/directorio compartido donde está esta bitácora y todo el material que si Dios quiere se irá generando en el desarrollo del Trabajo:

<https://drive.google.com/drive/folders/1o9MEYd09SzxtvS477htvN7O7dJdA6gUk?usp=sharing>

Esta URL se envía por Ideas para que todos tengamos acceso completo de edición a la carpeta/directorio compartido.

Como les comenté en la reunión, ayer jueves hubo devolución de materiales en otra asignatura y quedó disponible al menos desde ahora hasta si Dios quiere Agosto de 2026 una webcam.

En el directorio/carpeta compartida creé una (sub)carpeta para videos, donde dejo guardado el video que me enviaron con el primer experimento de procesamiento de video donde se ve que identifican movimiento en la RPi.

A partir de ahora, deberían ir volcando en esta bitácora lo que vayan probando y experimentando (sea exitoso o no) a la vez que poniendo al menos una carpeta por experimento para almacenar el código utilizado y si dejan registrado algún video o algún detalle más. Sé que uds. pueden usar otros métodos de almacenamiento y control de versiones, pero en mi caso necesito esta organización porque ya tengo demasiados proyectos, tesinas, tesis, trabajos de cátedra, etc. y no puedo mantener tanta heterogeneidad.

Mi sugerencia a corto plazo es que hagan un resumen de lo que entienden que va a ser su Trabajo de la asignatura (que creo que es lo que escribieron en Ideas), más allá de lo que está al principio como descripción, así lo dejamos de referencia a futuro.

15/12/25 - Profesor

Si les interesa, conseguí una Raspberry Pi Zero W que pueden usar estos próximos meses hasta Agosto 2026. Si no recuerdo mal, tiene la misma capacidad/funcionalidad que la Raspberry Pi3 B+, pero deberían confirmarlo uds. mismos. Para usarla, deberían conseguir una SD y todo lo necesario en términos de periféricos y alimentación, dado que lo que les puedo entregar es lo que está en la foto:

-Placa

-Cable/adaptador Micro USB - USB

-Cable/adaptador Micro HDMI - HDMI

(puse el billete de \$100 para que vean el tamaño)

Si les parece bien usarla, me tendrían que avisar por correo de la plataforma Ideas y acordar para que se las entregue.

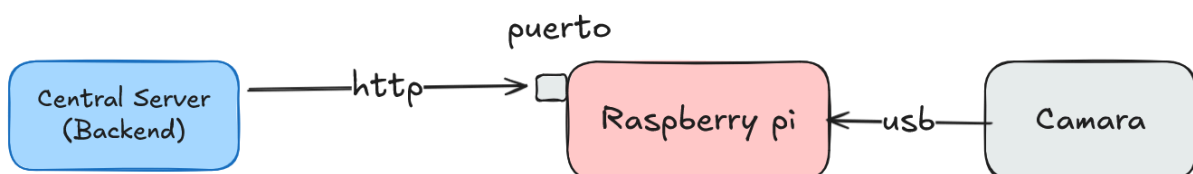


15/12/25 - Alumnos

Estuvimos analizando la propuesta del profesor de darnos la Raspberry Pi Zero W (mirar “bitácora 15/12/25 - Profesor”) y decidimos que es mejor quedarnos con la Raspberry Pi 4b físico que ya tenemos y luego agregar otros “edge” virtualizados.

Con respecto a los avances:

Esquema para prueba rapida de hardware



Este esquema es la primera versión de la implementación del sistema distribuido de la comunicación entre cámaras. El dispositivo edge físico es el conjunto de la Raspberry pi y la cámara.

El modelo de comunicación funciona de la siguiente manera:

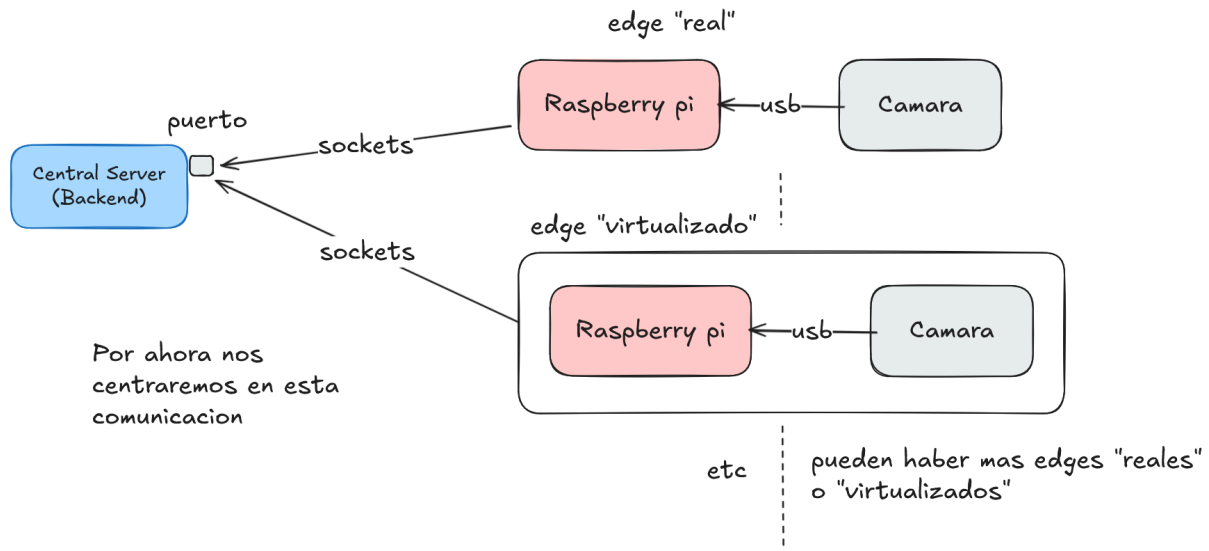
1. La raspberry pi se enciende y se conecta a la cámara de seguridad y recibe la información

de los frames de video, por otro lado, en el edge, se enciende un servidor de video con un puerto específico (ejemplo 5000) donde transmite los frames.

2. El servidor central se conecta a este puerto del edge y los visualiza

Luego, se decidió modificar el esquema anterior de la siguiente manera:

Siguiente esquema a implementar



Este esquema (pendiente de implementación), además de usar sockets, invierte el objeto que inicia la conexión. Al ser ahora los dispositivos edge quienes la inician, el sistema es mucho más modular, más seguro y evita tener que modificar el código al agregar un dispositivo edge nuevo.

16/12/25 - Profesor

Creo que esto es lo que ya conversamos, así que va bien. Comentarios extra/a futuro:

1.- Dejen comentado cómo/qué usan para los gráficos, así puedo recomendarlo a otros grupos, que algunas veces no saben cómo hacerlos.

2.- Para el trabajo propiamente dicho: el segundo esquema es básicamente el trabajo. Lo más importante es identificar y documentar el procesamiento en el edge/Raspberry cómo identificar el video "de interés" y en ese momento transmitir al que llaman "Central Server".

3.- Definan qué van a hacer en el "Central Server", implementen y documenten. Como conversamos, puede ser tan sencillo como dejar almacenado el video de los edges o hacer streaming al exterior o lo que uds. definan.

4.- Para el informe:

4.1) Redacten en voz pasiva, es decir en vez de "modificamos" ==> "se modificó". Si no les sale, pueden directamente pedir a una IA que les cambie la redacción y, por supuesto, uds. luego revisar que quede correcto y consistente.

4.2) Definan y mantengan consistente la terminología. Ej: "Raspberry Pi" y "edge" serían sinónimos, aunque "edge" sería más general. Pueden comentar al ppio. del informe algo así

como que son sinónimos aunque pueden haber muchas implementaciones diferentes de “edges” y que usaron al menos una implementación de edge con Raspberry Pi 4. Lo mismo para “Central Server” o “Back end”, etc.

5.- Como les comenté explícitamente por correo (copio):

“En caso de que sea necesario virtualizar algo, no lo vamos a hacer con docker sino con Vagrant y máquinas virtuales "completas", por decirlo de alguna manera, por algunas razones académicas y por otras técnicas. Es complicado explicar ambas por acá, así que lo dejaremos para una reunión/conversación.

Si quieren hacer experimentos con docker o con otro tipo de virtualización para las pruebas preliminares (como la que ya hicieron y subieron a la carpeta compartida), no hay problema, pero lo que entreguen como trabajo final debe ser con vagrant. Como en el caso de docker, lo que entregan será el vagrantfile y lo que uds. implementen/definan. Mi sugerencia es que hagan como en el TP2 de la asignatura (o al menos era la idea que hicieran en ese TP2): un vagrantfile y script/s que desplieguen automáticamente/programáticamente toda la parte del sistema completo que sea virtualizado, por decirlo de alguna manera.

06/01/26 - Alumnos

Hola profesor, esperamos que haya tenido felices fiestas y un buen comienzo de año. Estos días estuvimos un poco ocupados, pero ya hemos retomado el ritmo del proyecto.

Respondiendo su pregunta: Para los diagramas y esquemas estamos utilizando **Excalidraw**, el link es el siguiente: <https://excalidraw.com/>.

Hemos entendido los requerimientos, en caso de virtualizar utilizaremos la herramienta de vagrant.

Ya hemos hecho la primera implementación del Central Server basada en la nueva versión. En este caso el servidor escucha en el puerto 5555 y ahora es capaz de levantar un hilo por cada cámara que se conecta.

También hemos creado 2 nuevas clases: FrameManager y CameraHandler,

1. CameraHandler: Maneja la lógica de recepción de frames para cada cámara individual.
2. FrameManager: Estructura para acceder y guardar los datos específicos para cada cámara individual.

El código fue subido en la carpeta “TP - version 2”

30/01/26 - Alumnos

Continuando con el desarrollo del sistema distribuido, se ha realizado un avance significativos en la comunicación y visualización:

Implementación de Hilos Concurrentes (Threading): Se reestructuró el CentralServer para manejar múltiples tareas simultáneamente mediante el uso de hilos (Threads):

El CameraHandler ahora corre en su propio hilo para recibir el flujo de bytes desde los nodos edge sin bloquear el servidor principal.

Se creó una nueva clase WebServer que corre en un hilo separado. Este servidor levanta un servicio HTTP en el puerto 8081.

Visualización vía Streaming MJPEG: Para poder ver lo que capturan las cámaras en tiempo real, se implementó un protocolo de streaming MJPEG (Motion JPEG) en la clase WebServer. Esto permite que cualquier navegador web conectado a la red pueda visualizar el video accediendo a la URL del servidor (ej: http://IP_SERVIDOR:8081/stream?id=cam1), leyendo los frames almacenados en el FrameManager de manera segura (thread-safe).

Estado actual: La Raspberry Pi (Edge) captura video, lo procesa con OpenCV y lo envía al Servidor Central (PC Arch Linux). El servidor recibe el flujo y lo retransmite exitosamente a una interfaz web, logrando una tasa de refresco estable (~30 FPS).

Respecto al requerimiento de identificar personas, la implementación directa en el Edge está resultando compleja por limitaciones de hardware.

Intentamos implementar YOLOv5 Nano en la Raspberry Pi 4 para realizar detección de objetos específica. Sin embargo, utilizando solo la CPU del dispositivo, el rendimiento cayó drásticamente a unos ~5 FPS, lo cual es insuficiente para una aplicación de seguridad fluida.

Sin embargo todavía faltan hacer pruebas con modelos más actualizados y optimizados.

El código fue subido en la carpeta "TP - version 3"

02/02/26 - Profesor

Creo que tenemos que definir y cambiar varias cosas, entre ellas:

a) Todo el "empaquetamiento" creo que resta en vez de sumar, resta y agrega confusión y dependencia de un manejador de proyecto. Me parece mucho más claro y sencillo mantener 2

aplicaciones “planas”: una para cada ambiente. Quizás en PC no sería tanto problema, solo agregar “cáscara”, pero en Raspberry ese extra agrega procesamiento o, al menos almacenamiento, algo que en ningún edge se debe hacer. Por lo tanto, para ser “consistentes” tendríamos directamente dos aplicaciones sin empaquetar y compiláramos y ejecutaríamos tal como lo hicieron en las prácticas (o deberían haber hecho).

b) En el edge no se trata de identificar personas sino “movimiento”, luego vemos si vale la pena “personas”, por ahora “movimiento”. Calculo que va a ser relativamente “sencillo” lo de extender a “personas”, pero en cualquier caso, quedarnos con “movimiento” nos da factibilidad completa. Más específicamente: desde hace muchos años hay documentados proyectos con Raspberry Pi 3B+ en los cuales se identifica movimiento, por lo tanto debería ser “cortar y pegar” alguno de esos proyectos para este caso. Además, dado que están hechos en Raspberry Pi 3B+ sin dudas va a ser posible Raspberry Pi 4. Debo seguir aclarando que al menos conceptualmente ni siquiera la Raspberry Pi 3B+ es muy “indiscutiblemente” edge, pero por ahora es con lo que vamos a seguir.

c) Recuerden que en el “edge” hay dos “modos de funcionamiento”:

c.1) Sin haber detectado movimiento, todo lo que hay que hacer es procesar los frames para ver si se detecta movimiento. En este “modo de funcionamiento” no hay streaming al servidor, porque no tiene sentido, para esto está el edge: para enviar al servidor lo que SÍ tenga sentido que se procese en el servidor, eventualmente se provea a un operador humano. Podríamos llamar a este modo de funcionamiento “Modo Análisis”.

c.2) Una vez que se detectó movimiento, el “modo de funcionamiento” es lo que ya tienen: streaming al servidor, donde se hará otro procesamiento, se sirve el video al exterior, etc. Se podría llamar a este modo de funcionamiento “Modo Streaming”.

Se podrían pensar dos “disparadores” de cambio de estado/modo de funcionamiento: uno es el mencionado: si el edge está en “Modo Análisis” la identificación de movimiento cambia de estado a “Modo Streaming”. Para cambiar de “Modo Streaming” a “Modo Análisis” podríamos pensar en recibir alguna indicación desde el servidor, de manera tal que el disparador es directamente externo al edge y a partir de ese disparo el edge pasa a “Modo Análisis”.

Me parece que para aclarar vamos a tener que hacer una reunión en la Facultad, avísenme sus disponibilidades y acordamos día y hora.

18/02/26 - Alumnos

Hola Profesor,

Le respondemos sobre los puntos mencionados y los nuevos avances:

1. Aclaración sobre la arquitectura (Punto A):

Quería comentarle que el sistema siempre consistió en dos aplicaciones Java independientes (EdgeNode y CentralServer).

La referencia al "empaquetado" en el README se refería únicamente a la compresión del archivo (.jar) para su ejecución, no a una dependencia de software o estructura de proyecto compleja.

2. Modos de funcionamiento y Detección (Puntos B y C):

Estamos de acuerdo con el enfoque. De hecho, las pruebas con IA (YOLO) fueron experimentales para evaluar los límites de la Raspberry Pi 4.

Nuestras primeras versiones (v1) ya funcionaban mediante detección de movimiento, por lo que ahora hemos retomado esa misma lógica e implementado su enfoque:

- El Edge ahora opera en "Modo Análisis" (diferencia de frames) por defecto.
- Pasa a "Modo Streaming" solo al detectar cambios, funcionando correctamente y de manera fluida.
- El edge puede pasar a "Modo Análisis" de nuevo a nuestra voluntad, por ahora la implementación se hizo con un timer de 10 segundos, aunque puede ser un botón o cualquier otra lógica.

3. Entregables (Versión 4):

Hemos subido a la carpeta compartida "TP - version 4" el código fuente actualizado y en videos la demostración donde se ve el cambio de estados y la transmisión.

4. Tester:

Finalmente, le comento que conseguí el tester del que hablamos en la última reunión. Si le interesa utilizarlo o ver algo con él, puedo llevarlo cuando nos veamos o puede explicarme por este medio.

Pronto le comento la disponibilidad y horas para ir a la facultad

19/02/26 - Profesor

Háganme acordar que en la reunión les explique sobre:

- 1.- Por qué sigue siendo mala idea tener un .jar en este momento del desarrollo
- 2.- Por qué fue mala idea lo de IA-Límites de Raspberry 4
- 3.- Por qué no es buena idea poner expresiones como "implementado su enfoque"
- 4.- Al menos una forma de cambio de "Modo Streaming" a "Modo Análisis" y por qué

Buenísimo que van subiendo las versiones y aclarando qué se agrega/cambia en cada caso.

No recuerdo lo del tester... llévalo a la reunión y háganme acordar de qué se trata.

20/02/26 - Reunión

Se aclararon los puntos 1 a 4 de los comentarios anteriores (19/2/26).

Quedó para ver más adelante, cuando esté el sistema más completo operativo si vale la pena dedicar tiempo al consumo en la RPi 4 con el tester.

Se aclararon algunos detalles de la distribución de operación del servidor al que los “edges” con cámara harían streaming. Quedó una foto del pizarrón al respecto, que agregarán los estudiantes.

Se conversaron algunos detalles de interfaz de usuario, en principio menores, pero a la vez “separados” de la operación del servidor/controlador de los edges/RPis.

Deberían haber al menos 2 edges y al menos uno de ellos la RPi. En caso de no haber 2 RPis se virtualiza el 2do edge. Si tienen recursos podrían ver cuántos llegan a virtualizar sin poner en riesgo el funcionamiento del sistema completo.

17/03/26 - Alumnos

Se detalla a continuación el avance correspondiente a la Versión 5 del sistema, donde se implementó la lógica de control de estados y la comunicación bidireccional entre los nodos Edge y el Central Server.

Arquitectura y Control de Estados

Se estructuró el funcionamiento de los nodos Edge bajo una máquina de estados estrictamente excluyente: el nodo analiza el video localmente o transmite el flujo al servidor central, garantizando que nunca se realicen ambas tareas en simultáneo para no superponer carga de procesamiento en el hardware.

Modo Análisis (Estado inicial/por defecto): El Edge procesa localmente los frames buscando movimiento. El Central Server se mantiene a la espera sin recibir video.

Modo Streaming: Al detectar movimiento localmente, el Edge detiene su análisis por software y se dedica únicamente a transmitir el video vía sockets TCP al Central Server. En este estado, el Central Server asume el procesamiento de los frames recibidos.

Transiciones de Estado y Comunicación

Para gestionar los cambios de estado (Modo Análisis <-> Modo Streaming), se definieron los siguientes mecanismos, separando el flujo de video del flujo de comandos:

Transición Automática por Detección: El Edge inicia la transmisión de video por socket TCP de manera autónoma al identificar movimiento local.

Transición Automática por Inactividad: Si se transcurre un tiempo configurado en segundos sin registrar actividad nueva, el Central Server envía una petición HTTP al Edge para forzar el retorno al Modo Análisis y detener la transmisión.

Transición Forzada (Comandos Web): Se incorporaron comandos HTTP para forzar el encendido o apagado del flujo de video bajo demanda.

Implementación Técnica

Central Server: Se dividió en dos servidores independientes. Un TCP Server escuchando en el puerto 5555 exclusivamente para recibir las conexiones de video, y un Web Server que expone la API de comandos (/api/start, /api/stop, /api/cameras) y rutea el streaming hacia los clientes web.

Edge Node: Además del cliente TCP para enviar video, se levantó un servidor HTTP local (CommandServer) en el puerto 8080 que escucha las instrucciones remotas (/start, /stop) para actualizar su estado interno (EdgeStateManager).

Web Client: Para facilitar los comandos y transmisión, se creó una interfaz visual, basada en tecnologías HTML, CSS, usando como framework React. El mismo utiliza los comandos que expone el Central Server.

El código fuente actualizado fue subido a la carpeta "TP - version 5".

También, se adjuntó un video demostrativo con el funcionamiento de esta arquitectura en la carpeta correspondiente.

8/04/26 - Reunión

Reemplazar en el edge el “servidor HTTP” por una conexión TCP

Todo el funcionamiento está bien, queda como en la v5

Entrega:

- Documento con explicación del sistema completo en .pdf
 - Funcionamiento en general
 - Lo que está a continuación de cada parte, dado que tanto el servidor como el edge se virtualizan, en la instalación se pone el código fuente y en la parte de aprovisionamiento de cada virtual estará la compilación.

- Virtualizar el Central Server. Esto implica definir dentro de un vagrantfile todo lo que estará en este servidor y quedará autocontenido, tanto para la entrega como para una eventual instalación real. Documentar:
 - Completo el vagrantfile que le corresponde, donde quedarán explícitas las dependencias de todo lo que se instale.
 - En particular en la sección de aprovisionamiento se instalará el código y se compilará. Si quieren, agrega que la parte de código no es “real” en el sentido de un deploy, sino que es para esta entrega.
 - La virtualización del edge. Idem servidor. Con lo que se entrega para la virtualización del edge, estarán entregando los fuentes que corren en los edges y la forma en que se instala.
- Video explicando la funcionalidad... no más de 5 min. recortar sin problemas
- Bitácora en .pdf (tal como está)
- El propio .tar/zip/etc.